

Working with Abstraction

Architecture, Refactoring and Enumerations



SoftUni Team
Technical Trainers
Software University
<http://softuni.bg>



```
class Car
{
    public string model;
    public double fuelAmount;
    public double fuelCostForPerKm;
    public int distanceTraveled;

    public Car(string model, double fuelAmount)
    {
        this.model = model;
        this.fuelAmount = fuelAmount;
        this.fuelCostForPerKm = fuelCost;
        this.distanceTraveled = 0;
    }
}
```



Best Practices

Table of Contents

1. Project Architecture

- Methods
- Classes
- Projects

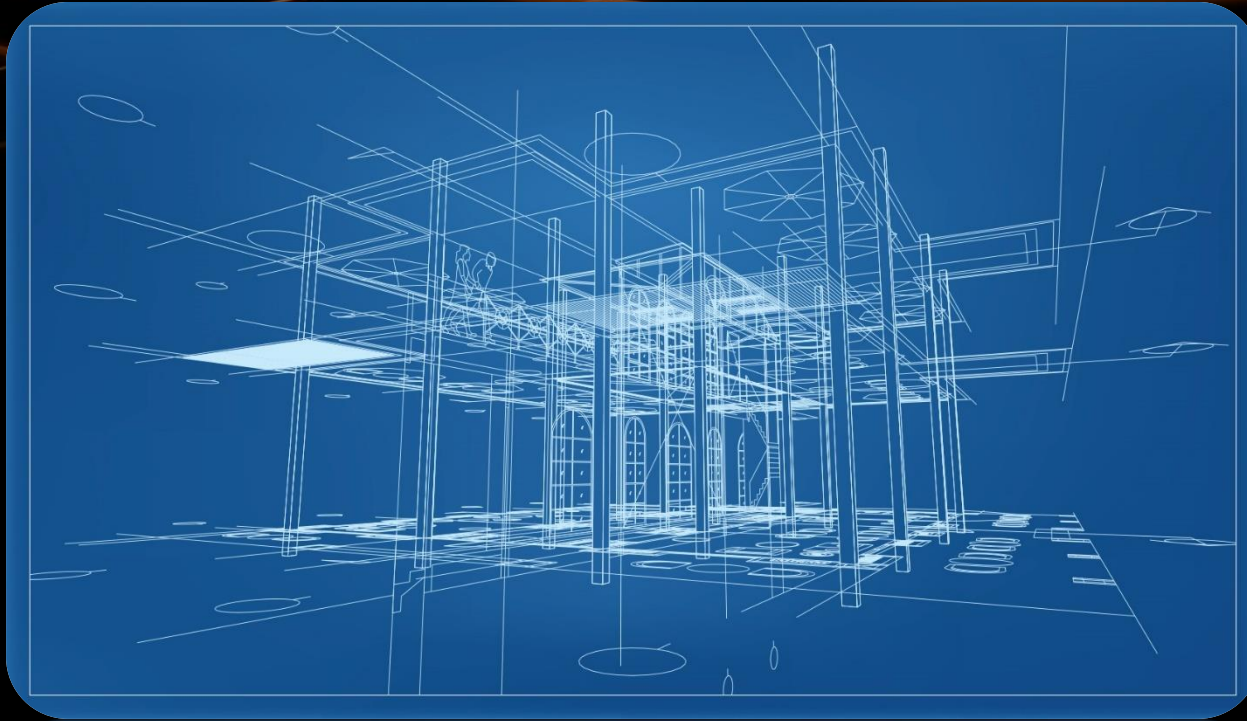
2. Code Refactoring

3. Enumerations



sli.do

#CSharp-Fund



Project Architecture

Splitting Code into Logical Parts

Splitting Code into Methods

- We use **methods** to split code into **functional blocks**
 - Improves **code readability**
 - Allows for **easier debugging**

```
foreach (char move in moves)
{
    for (int row = 0;
        row < room.Length; row++)
    {
        for (int col = 0;
            col < room[row].Length; col++)
        {
            if (room[row][col] == 'b')
            ...
        }
    }
}
```



```
foreach (char move in moves)
{
    MoveEnemies();
    KillerCheck();
    MovePlayer(move);
    ...
}
```



Splitting Code into Methods (2)

- Methods let us easily reuse code
- We change the method once to affect all calls

```
var bankAcc = new BankAccount();  
bankAcc.Id = 1;  
bankAcc.Deposit(20);  
Console.WriteLine($"Account {bankAcc.Id}, balance {bankAcc.Balance}");  
bankAcc.Withdraw(10);  
...  
Console.WriteLine($"Account {bankAcc.Id}, balance {bankAcc.Balance}");
```



Override `.ToString()` to set a global printing format

```
Console.WriteLine(bankAcc.ToString());
```

Splitting Code into Methods (3)

- A single method should complete a **single task**

```
void Withdraw ( ... )  
void Deposit ( ... )  
decimal GetBalance ( ... )  
string ToString ( ... )
```



```
void DoMagic ( ... )  
void DepositOrWithdraw ( ... )  
decimal DepositAndGetBalance ( ... )  
string ParseDataAndReturnResult ( ... )
```



Problem: Rhombus of Stars

- Draw on the console a rhombus of stars with size **n**

n = 3



```
  *  
 * *  
* * *  
 * *  
  *
```

n = 2



```
 *  
* *  
  *
```

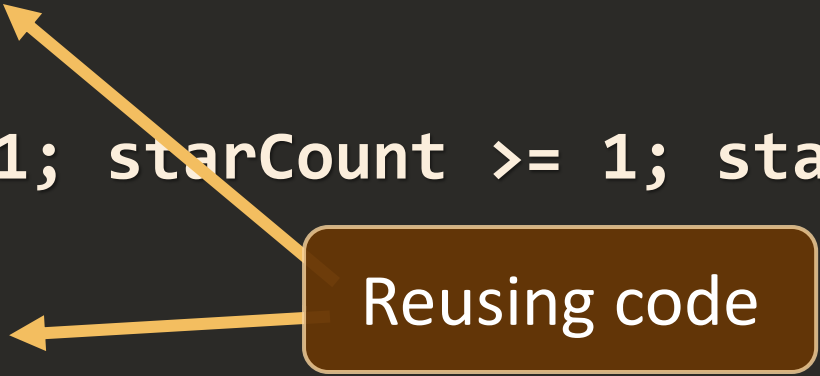
n = 1



```
*
```


Solution: Rhombus of Stars

```
static void Main()
{
    int size = int.Parse(Console.ReadLine());
    for (int starCount = 1; starCount <= size; starCount++)
    {
        PrintRow(size, starCount);
    }
    for (int starCount = size - 1; starCount >= 1; starCount--)
    {
        PrintRow(size, starCount);
    }
}
```



Solution: Rhombus of Stars (2)

```
static void PrintRow(int figureSize, int starCount)
{
    for (int i = 0; i < figureSize - starCount; i++)
    {
        Console.Write(" ");
    }
    for (int col = 1; col < starCount; col++)
    {
        Console.Write("* ");
    }
    Console.WriteLine("*");
}
```

Splitting Code into Classes

- Just like methods, classes should not know or do too much

```
int[] numbers = args.Split().Select(int.Parse).ToArray();  
...  
Console.Write(string.Join(" ", numbers));
```



```
GodMode master = new GodMode();  
int[] numbers = master.ParseAny(args);  
...  
int[] numbers2 = master.CopyAny(numbers);  
master.PrintToConsole(master.GetDate());  
master.PrintToConsole(numbers);
```

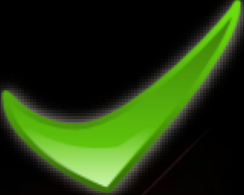
Splitting Code into Classes (2)

- We can also break our code up logically into **classes**

```
int[] numbers = args.Split().Select(int.Parse).ToArray();  
.  
.  
Console.WriteLine(string.Join(" ", numbers));
```



Hiding
implementation



```
ArrayParser parser = new ArrayParser();  
OuputWriter printer = new OuputWriter();  
int[] numbers = parser.IntegersParse(args);  
int[] coordinates = parser.IntegerParse(args1);  
printer.PrintToConsole(numbers);
```

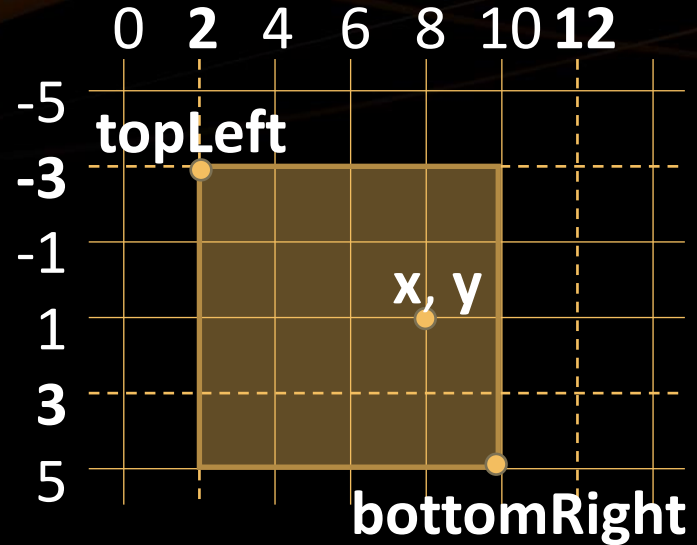
Allow us to
change output
destination

Helps us avoid
repeating code

Problem: Point in Rectangle

- Create a **Point** class holding the horizontal and vertical coordinates.
- Create a **Rectangle** class that holds one of the sides (that's your abstraction, **you don't need more**).
- Add **Contains** method in the **Rectangle** class that takes a **Point** as an argument and **returns if it's inside** the current object of the **Rectangle** class.

Solution: Point in Rectangle



```
public class Point
{
    private int x;
    private int y;
    //TODO: Public properties
}
```

```
public class Rectangle
{
    private Point topLeft;
    private Point bottomRight;

    //TODO: Public properties
    public bool Contains(Point point)
    {
        //TODO: Implement
    }
}
```

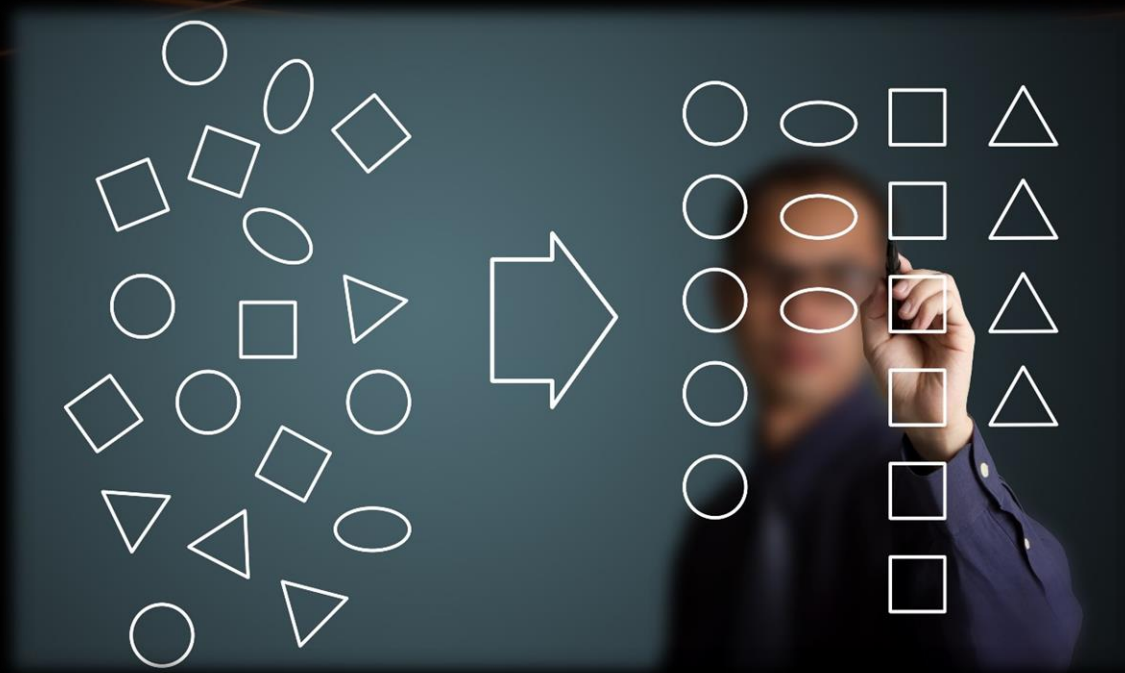
Solution: Point in Rectangle (2)

```
public bool Contains(Point point)
{
    bool isInHorizontal =
        this.TopLeft.X <= point.X &&
        this.BottomRight.X >= point.X;

    bool isInVertical =
        this.TopLeft.Y <= point.Y &&
        this.BottomRight.Y >= point.Y;

    bool isInRectangle = isInHorizontal && isInVertical

    return isInRectangle;
}
```



Refactoring

Restructuring and Organizing Code

Refactoring

- Restructuring code without changing the **behaviour**
 - Improves code **readability**
 - Reduces **complexity**

```
class ProblemSolver { public static void DoMagic() { ... } }
```

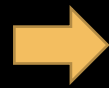


```
class CommandParser { public static Func<T, T> ParseCommand(); }  
class DataModifier { public static T Execute(); ... }  
class OutputFormatter { public static void Print(); ... }
```

Refactoring Techniques

- Componentization - breaking code apart reusable units
 - Extracting parts of **methods** and **classes** into **new ones**

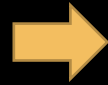
```
DepositOrWithdraw()
```



```
Deposit()  
Withdraw()
```

- Improving names of **variables**, **methods**, **classes**, etc.

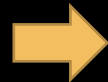
```
string str;
```



```
string name;
```

- Moving **methods** or **fields** to **more appropriate classes**

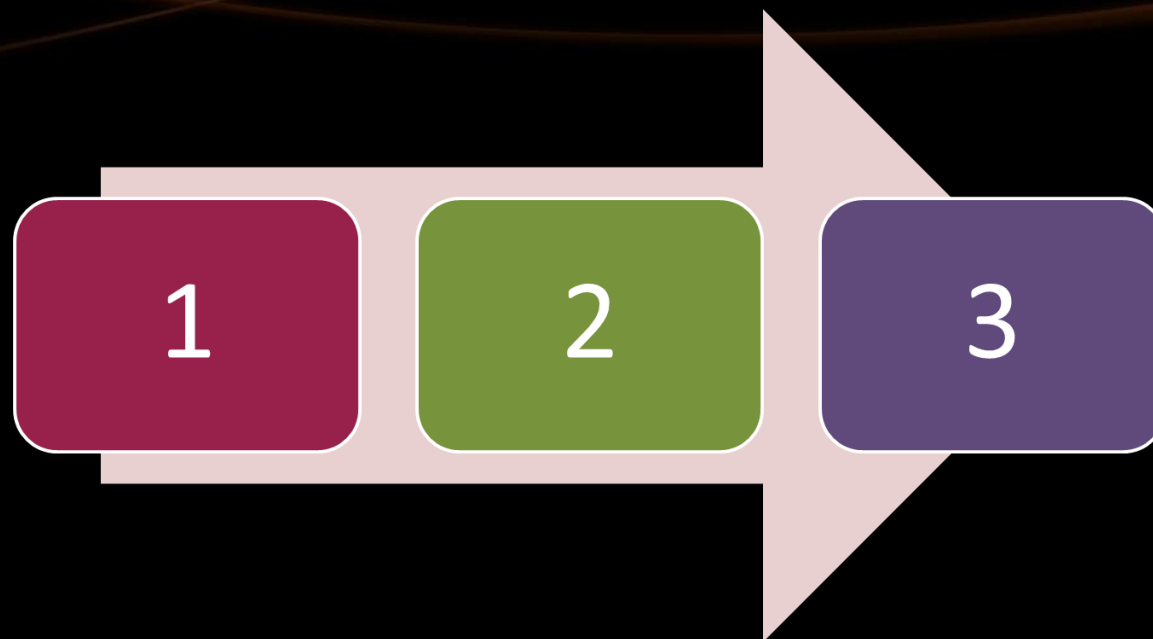
```
Car.Open()
```



```
Door.Open()
```

Problem: Student System

- You are given a **working** Student System project to **refactor**
- **Break it up** into smaller functional units and make sure it **works**
- It supports the following **commands**:
 - **"Create <studentName> <studentAge> <studentGrade>"** – creates a new student.
 - **"Show <studentName>"** – prints information about a student
 - **"Exit"** – closes the program.



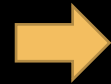
Enumerations

Syntax and Usage

Enumerations

- Enumerations represent a numeric value from a fixed set as text
- We can use them to pass **arguments** to **methods** without making code confusing

```
GetDailySchedule(0)
```



```
GetDailySchedule(Day.Mon)
```

```
enum Day { Mon, Tue, Wed, Thu, Fri, Sat, Sun }
```

Use the **enum** keyword

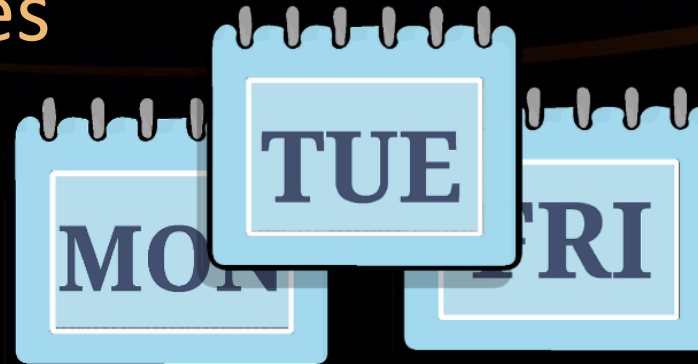
By default, enums start at 0

Every next value is incremented by 1

Enumerations (2)

- We can customize enum values

```
enum Day
{
    Mon = 1,
    Tue,      //2
    Wed,      //3
    Thu,      //4
    Fri,      //5
    Sat,      //6
    Sun       //7
}
```



```
enum CoffeeSize
{
    Small = 100,
    Normal = 150,
    Double = 300
}
```



Problem: Hotel Reservation

- Create a class **PriceCalculator** that calculates the total price of a holiday, given the **price per day**, **number of days**, the **season** and a **discount type**
- The **discount type** and **season** should be **enums**
- The **price multipliers** will be:
 - 1x for Autumn, 2x for Spring, etc.
- The **discount types** will be:
 - None – 0%
 - SecondVisit – 10%
 - VIP – 20%



Solution: Hotel Reservation

```
public enum Discount
{
    None,
    SecondVisit = 10,
    VIP = 20
}

public enum Season
{
    Spring = 2,
    Summer = 4,
    Autumn = 1,
    Winter = 3
}
```



Solution: Hotel Reservation (2)

- Now, use the **class** like this:
 - Read **input** from the console
 - Parse it and call the `CalculatePrice()` method

50.25 5 Summer VIP



804.00

40 10 Autumn SecondVisit



360.00

120.20 2 Winter



721.20

The discount input
may be omitted!



Solution: Hotel Reservation (2)

```
public decimal CalculatePrice (decimal pricePerDay,  
    int numberOfDays, Season season, Discount discount)  
{  
    int multiplier = (int)season;  
    decimal discountMultiplier = (decimal)discount / 100;  
    decimal priceBeforeDiscount =  
        numberOfDays * pricePerDay * multiplier;  
    decimal discountedAmount =  
        priceBeforeDiscount * discountMultiplier;  
    decimal finalPrice =  
        priceBeforeDiscount - discountedAmount;  
    return finalPrice;  
}
```

We can cast
enums to
numeric values

Summary

- Well organized code is easier to work with
- We can reduce complexity using
 - Methods
 - Classes
 - Projects
- We can **refactor** existing code by
 - Breaking code apart
 - Improving **names** and **locations**



Summary (2)

- Enumerations define a fixed set of constants
 - E.g. the days of the week
- They represent numeric values with more readability
- We can easily cast enums to numeric data types



Working with Abstraction



Questions?



License

- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license



- Attribution: this work may contain portions from
 - "Fundamentals of Computer Programming with C#" book by Svetlin Nakov & Co. under CC-BY-SA license
 - "C# Part I" course by Telerik Academy under CC-BY-NC-SA license
 - "C# Part II" course by Telerik Academy under CC-BY-NC-SA license

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



**Software
University**

